

Apresentacao explicada do projeto Ride Challenge

Este documento foi refeito em formato de explicacao corrida. A proposta desta versao e evitar topicos soltos e transformar o conteudo em uma fala detalhada, como se voce estivesse explicando o projeto para uma pessoa que esta começando agora e, aos poucos, levando essa pessoa ate um nivel mais avancado de entendimento. Os capitulos continuam existindo para organizar o estudo, mas dentro de cada capitulo a explicacao foi escrita em paragrafos, com começo, meio e fim.

Voce pode usar este material de duas formas. A primeira forma e estudar lendo em ordem, porque ele começa explicando o que o sistema faz e depois entra nas tecnologias, nos servicos, nos arquivos e nos fluxos. A segunda forma e usar como roteiro de apresentacao, porque muitas partes ja estao escritas como frases que voce pode adaptar para falar diante da banca.

O projeto Ride Challenge e uma aplicacao full stack que simula um fluxo simplificado de corridas parecido com um aplicativo de transporte. Existe uma parte visual, feita em Angular, onde o usuario entra como cliente ou motorista. Existe uma parte de backend, feita em Java com Spring Boot e Spring Cloud, onde ficam as regras de negocio, as APIs, os microservicos e as integracoes com banco, fila, cache e tempo real. O cliente cria uma corrida informando origem e destino. O motorista visualiza corridas disponiveis em tempo real e pode aceitar uma corrida. Quando isso acontece, o status muda, o dado e salvo e a tela e atualizada.

Uma forma segura de abrir a apresentacao e dizer que este projeto nao e apenas um cadastro simples. Ele foi construido para demonstrar integracao entre varias partes de um sistema moderno. O Angular cuida da interface. O API Gateway centraliza a entrada. O Eureka ajuda os servicos a se encontrarem. O Config Server centraliza configuracoes. O Account Service cuida das contas. O Ride Service cuida das corridas. O PostgreSQL salva os dados. O Kafka transporta eventos. O Redis guarda status em cache. O WebSocket atualiza a tela do motorista em tempo real. O Docker Compose sobe tudo junto para facilitar a execucao e a demonstracao.

Capítulo 1 - O que o projeto faz na pratica

Para explicar o projeto desde o nível iniciante, comece contando a historia do usuario. Imagine que uma pessoa abre a aplicacao no navegador. Ela escolhe se quer entrar como cliente ou como motorista. Se entrar como cliente, consegue criar uma corrida informando uma origem e um destino. Essa solicitacao sai do navegador, passa pelo frontend Angular, chega ao backend pelo API Gateway e e processada pelo Ride Service. O Ride Service valida se o cliente existe, cria a corrida, salva no banco de dados e publica um evento no Kafka dizendo que uma nova corrida foi criada.

Depois que esse evento e publicado, outra parte do Ride Service escuta a fila Kafka. Quando a mensagem chega, o sistema envia uma notificacao por WebSocket para os motoristas conectados. Isso significa que a tela do motorista nao precisa ficar atualizando manualmente o tempo todo. Quando uma nova corrida aparece, o servidor avisa o navegador. Esse comportamento e importante porque aproxima o projeto de um sistema em tempo real.

Quando o motorista aceita uma corrida, acontece outro fluxo. O frontend do motorista envia uma requisicao para o backend dizendo qual corrida ele quer aceitar e qual e o id do motorista. O Ride Service consulta o Account Service para confirmar se aquele usuario realmente e motorista. Depois busca a corrida no banco e verifica se ela ainda esta disponivel. Se estiver, atribui o motorista, muda o status para em andamento, salva no PostgreSQL, grava o status no Redis e envia uma notificacao para atualizar as telas.

No nível iniciante, voce pode resumir dizendo que o projeto permite criar usuarios, criar corridas, mostrar corridas para motoristas e aceitar uma corrida. No nível intermediario, voce explica que esses passos passam por frontend, gateway, microservicos, banco, Kafka, Redis e WebSocket. No nível avancado, voce mostra que o projeto foi separado em camadas para reduzir acoplamento, facilitar testes e deixar cada tecnologia com uma responsabilidade clara.

A principal mensagem deste capítulo e que cada parte do projeto existe para resolver um problema especifico. O frontend resolve a interacao com o usuario. A API resolve a comunicacao entre navegador e servidor. O banco resolve a persistencia. A fila resolve comunicacao assincrona. O cache melhora consulta de status. O WebSocket resolve tempo real. Os testes ajudam a garantir que as regras funcionem.

Capítulo 2 - Como uma requisicao percorre o sistema

Para entender o projeto, e muito importante entender o caminho de uma requisicao. Uma requisicao e um pedido que o navegador faz para o backend. Quando o usuario clica em um botao no Angular, como criar conta ou criar corrida, o componente da tela chama um service do frontend. Esse service usa o HttpClient do Angular para enviar uma chamada HTTP para o endereco base da API, que no projeto e `http://localhost:8080`.

Esse endereco nao aponta diretamente para o Account Service nem diretamente para o Ride Service. Ele aponta para o API Gateway. O Gateway funciona como a porta principal de entrada. Quando a chamada chega em uma rota que comeca com `/accounts`, o Gateway encaminha para o Account Service. Quando a chamada chega em uma rota que comeca com `/rides`, o Gateway encaminha para o Ride Service. Quando a chamada e de WebSocket em `/ws`, o Gateway tambem encaminha para o Ride Service, mas usando a rota apropriada para WebSocket.

O Gateway nao precisa decorar um IP fixo de cada servico. Ele usa Eureka para descobrir onde os servicos estao. Isso e um conceito importante em microservicos. Em vez de um servico depender de endereco fixo, cada servico se registra em um catalogo, e os outros servicos conseguem localiza-lo pelo nome. No projeto, os nomes importantes sao ACCOUNT-SERVICE e RIDE-SERVICE.

Depois que a requisicao chega ao servico correto, o controller recebe os dados HTTP e transforma isso em um comando de aplicacao. Por exemplo, uma chamada para criar corrida chega em RideController, que monta um CreateRideCommand. O command e passado para o caso de uso DefaultCreateRideUseCase. O caso de uso executa a regra de negocio, conversa com contratos do dominio e devolve um output. Depois o controller transforma esse output em uma response JSON.

Essa sequencia mostra uma decisao arquitetural importante. O controller nao deve conter a regra principal. Ele e uma camada de entrada. A regra principal fica no caso de uso e na entidade de dominio. Isso deixa o codigo mais organizado e facilita testes, porque e possivel testar o caso de uso sem subir servidor HTTP.

Capítulo 3 - A arquitetura do backend

O backend fica dentro da pasta `ride-challenge-backend`. Ele foi organizado como um conjunto de microsserviços. Um microsserviço é uma aplicação menor, com uma responsabilidade específica, que trabalha junto com outras aplicações. Em vez de colocar tudo em um único backend grande, o projeto separa configuração, descoberta de serviços, entrada da API, contas e corridas.

O Config Server é o serviço que centraliza configurações. Ele roda na porta 8888 e entrega arquivos YAML para os outros serviços. Isso permite que as configurações fiquem organizadas em um lugar só. O Eureka Server é o serviço de descoberta. Ele roda na porta 8761 e funciona como um catálogo onde os outros serviços se registram. O API Gateway roda na porta 8080 e é a entrada externa do sistema. O Account Service cuida de contas de clientes e motoristas. O Ride Service cuida do fluxo principal de corridas.

Além dos serviços Java, o ambiente tem PostgreSQL, Kafka, Redis e frontend. O PostgreSQL é o banco relacional que salva contas e corridas. O Kafka é a fila de mensagens usada quando uma corrida é criada. O Redis é o cache usado para status de corrida. O frontend Angular é servido em `localhost:4200` quando o projeto roda via Docker Compose.

Essa arquitetura permite mostrar várias habilidades ao mesmo tempo. Ela mostra que você sabe construir APIs REST, sabe separar responsabilidades, sabe trabalhar com banco de dados, sabe usar comunicação assíncrona, sabe usar cache, sabe entregar tempo real e sabe empacotar tudo com Docker. Ao apresentar, deixe claro que essa complexidade foi usada para fins didáticos e para aproximar o desafio de uma arquitetura real. Em um sistema pequeno, algumas partes poderiam ser mais simples, mas no contexto do projeto cada tecnologia demonstra um conceito importante.

No nível avançado, o ponto mais forte da arquitetura é a separação entre regra de negócio e infraestrutura. As entidades e casos de uso não ficam presos diretamente ao PostgreSQL, Kafka, Redis ou WebSocket. Eles dependem de interfaces, como `RideGateway`, `AccountClient`, `SentEventService`, `RideStatusCache` e `DriverNotifier`. As implementações concretas ficam na infraestrutura. Isso é uma forma de reduzir acoplamento e deixar o sistema mais testável.

Capítulo 4 - Java 17 explicado no projeto

Java 17 é a linguagem usada em todo o backend. Em termos simples, Java é responsável pela parte que roda no servidor. É no código Java que o sistema recebe pedidos do frontend, valida informações, aplica regras de negócio, salva dados e integra com outras tecnologias. A escolha de Java faz sentido porque ele é muito usado em sistemas corporativos, tem uma comunidade grande, possui ferramentas maduras e combina muito bem com Spring Boot.

No projeto, Java aparece em classes, interfaces, enums e records. Uma classe representa um objeto ou uma parte do sistema com comportamento. A classe `Account` representa uma conta. A classe `Ride` representa uma corrida. Uma interface representa um contrato, ou seja, ela diz o que precisa existir, mas não diz exatamente como será feito. `RideGateway` é uma interface que diz que deve ser possível salvar e buscar corridas. A implementação concreta desse contrato é `RidePostgresGateway`, que usa PostgreSQL.

Os enums aparecem quando o sistema precisa limitar valores aceitos. `AccountType` define os tipos `CLIENT` e `DRIVER`. `RideStatus` define os estados `CREATED`, `IN_PROGRESS`, `COMPLETED` e `CANCELLED`. Isso evita que o sistema aceite textos aleatórios como status ou tipo de conta. Em vez de depender de strings soltas, o código trabalha com valores controlados.

Os records são usados para objetos simples de entrada e saída. `CreateRideCommand` carrega os dados necessários para criar corrida. `CreateRideOutput` carrega o id retornado. `RideCreatedMessage` carrega a mensagem publicada no Kafka. O record é útil porque reduz código repetitivo, já que o Java gera automaticamente construtor, acessores e outros métodos básicos.

No nível avançado, Java ajuda na organização por contratos. O domínio define o que precisa através de interfaces, e a infraestrutura implementa. Isso facilita testes, porque em um teste unitário você pode trocar o banco real por um mock. Também facilita manutenção, porque se um dia o projeto trocasse PostgreSQL por outro banco, a regra principal poderia continuar dependendo de `RideGateway`.

Capítulo 5 - Spring Boot explicado no projeto

Spring Boot é o framework que facilita criar os microsserviços Java. Sem Spring Boot, seria necessário configurar manualmente servidor web, rotas HTTP, injeção de dependências, validação, conexão com banco, integração com Kafka, Redis e WebSocket. O Spring Boot entrega uma base pronta e permite que o desenvolvimento foque mais nas regras do projeto.

Os arquivos `AccountServiceApplication`, `RideServiceApplication`, `ConfigServerApplication`, `EurekaServerApplication` e `ApiGatewayApplication` são pontos de entrada dos serviços. Eles normalmente possuem a anotação `SpringBootApplication` e um método `main`. O método `main` chama `SpringApplication.run`, que inicializa a aplicação, carrega configurações, cria objetos gerenciados e sobe o servidor.

As anotações do Spring aparecem em várias partes. `RestController` indica que uma classe recebe chamadas HTTP. `Service` indica que uma classe é um serviço gerenciado pelo Spring. `Component` registra uma classe genérica no contexto. `Configuration` indica uma classe de configuração. `Bean` indica que um método cria um objeto que o Spring deve administrar. Essas anotações evitam a criação manual de objetos e permitem que o Spring resolva dependências automaticamente.

Um conceito central é injeção de dependências. Em vez de uma classe criar tudo que precisa com `new`, ela declara no construtor suas dependências. Por exemplo, `DefaultCreateRideUseCase` precisa de `RideGateway`, `AccountClient` e `SentEventService`. Ele não cria essas dependências diretamente. O Spring monta o grafo de objetos e injeta a implementação correta. Isso deixa o código mais flexível e testável.

Na apresentação, você pode dizer que Spring Boot foi escolhido porque acelera o desenvolvimento de APIs e microsserviços, integra facilmente com outras tecnologias e permite separar responsabilidades usando controllers, services, repositories, configurações e beans.

Capítulo 6 - Maven e os arquivos pom.xml

Maven é a ferramenta que gerencia o backend Java. Ele baixa dependências, compila o código, roda testes e empacota as aplicações. O arquivo pom.xml é o arquivo onde cada projeto declara seu nome, versão, dependências e plugins de build. Pense no pom.xml como a ficha técnica de cada microserviço.

Na raiz do backend existe um pom.xml com packaging pom. Isso significa que ele não gera uma aplicação sozinho. Ele organiza os módulos config-server, eureka-server, api-gateway, account-service e ride-service. Cada módulo tem seu próprio pom.xml, porque cada serviço tem dependências específicas. O Account Service precisa de web, JPA, validação, Config Client, Eureka Client, PostgreSQL e testes. O Ride Service precisa dessas dependências e também de OpenFeign, Kafka, Redis e WebSocket, porque ele tem mais integrações.

O Maven também organiza os testes. Testes unitários rodam na fase test. Testes de integração rodam na fase integration-test com o plugin Failsafe. A fase verify executa o build completo e valida o gate de cobertura. O JaCoCo mede cobertura de testes e foi configurado para exigir pelo menos noventa por cento de cobertura em domínio e aplicação. Essa decisão mostra preocupação com qualidade justamente onde ficam as regras mais importantes.

Na apresentação, explique que Maven evita instalar bibliotecas manualmente. Quando o projeto declara uma dependência, o Maven baixa a versão correta e monta o classpath. Isso padroniza o ambiente de build e facilita rodar o projeto em outra máquina ou no GitHub Actions.

O ponto avançado é que cada serviço declara apenas o que precisa. Isso evita dependências desnecessárias. O Account Service não precisa de Kafka porque ele não publica nem consome eventos. O Ride Service precisa, porque cria corrida e dispara notificações baseadas em evento. Esse detalhe mostra que a divisão por serviços também aparece no gerenciamento de dependências.

Capítulo 7 - Config Server explicado

O Config Server é um microserviço responsável por centralizar configurações. Em projetos com vários serviços, cada aplicação precisa saber porta, nome, banco, endereço do Eureka, endereço de Kafka, Redis e outras informações. Se cada serviço guardasse tudo isolado, ficaria mais difícil manter. O Config Server resolve isso oferecendo configurações a partir de um lugar central.

No arquivo `ConfigServerApplication.java`, a anotação `EnableConfigServer` transforma aquela aplicação em um servidor de configuração. O `application.yaml` do próprio Config Server define que ele roda na porta 8888, usa perfil `native` e procura arquivos de configuração dentro de `classpath:/configurations`. Isso significa que os YAMLS dos serviços ficam empacotados dentro do próprio Config Server.

Dentro da pasta `configurations` existem arquivos como `account-service.yaml`, `ride-service.yaml`, `api-gateway.yaml` e `eureka-server.yaml`. O `account-service.yaml` define a porta 8081, a conexão com o banco `account_db` e o endereço do Eureka. O `ride-service.yaml` define a porta 8082, a conexão com `ride_db`, Redis, Kafka e configurações de `timeout`. O `api-gateway.yaml` define rotas e CORS. O `eureka-server.yaml` define a porta 8761 e informa que o próprio Eureka não precisa se registrar em si mesmo.

Também existem arquivos com sufixo `docker`. Eles ajustam endereços quando os serviços rodam dentro de containers. Fora do Docker, é comum usar `localhost`. Dentro do Docker, um container não acessa outro usando `localhost`, porque `localhost` seria ele mesmo. Por isso, no perfil Docker, o Ride Service usa nomes como `postgres`, `redis` e `kafka`. Esse detalhe é importante para demonstrar que você entende diferença entre ambiente local e ambiente em containers.

Na apresentação, diga que o Config Server deixa a configuração fora do código Java. Assim, alterar endereço de banco, porta ou broker não exige mudar a regra de negócio. Isso é uma prática comum em sistemas distribuídos.

Capítulo 8 - Eureka Server explicado

Eureka Server é usado para service discovery, que significa descoberta de serviços. Em uma arquitetura de microsserviços, os serviços precisam se encontrar. O API Gateway precisa encontrar o Account Service e o Ride Service. O Ride Service precisa chamar o Account Service. Em vez de depender de IP fixo, os serviços se registram no Eureka e podem ser localizados pelo nome.

O arquivo `EurekaServerApplication.java` é a classe principal do serviço. A anotação `EnableEurekaServer` habilita o comportamento de servidor Eureka. O arquivo `eureka-server.yaml` define que ele roda na porta 8761. Também define `register-with-eureka` como `false` e `fetch-registry` como `false`, porque o próprio servidor Eureka não precisa se registrar nele mesmo nem buscar uma lista de outros serviços para funcionar como servidor.

Os serviços clientes, como Account Service, Ride Service e API Gateway, apontam para o Eureka usando a propriedade `eureka.client.service-url.defaultZone`. Quando sobem, eles se registram e passam a aparecer no catálogo. O Gateway então consegue usar rotas como `lb://ACCOUNT-SERVICE` e `lb://RIDE-SERVICE`. O prefixo `lb` indica que a chamada será resolvida com balanceamento e descoberta de serviço.

Mesmo que no projeto exista apenas uma instância de cada serviço, a arquitetura já mostra um conceito usado em ambientes maiores. Se existissem várias instâncias do Ride Service, o mecanismo de descoberta permitiria distribuir chamadas entre elas. Para a apresentação, você pode explicar que Eureka reduz dependências de endereço fixo e torna a comunicação entre microsserviços mais flexível.

O ponto principal é que Eureka não processa regra de corrida nem salva dados. Ele existe para ajudar os serviços a se encontrarem. Essa é a responsabilidade dele dentro da arquitetura.

Capítulo 9 - API Gateway explicado

O API Gateway é a entrada única da API. O frontend Angular não chama diretamente Account Service na porta 8081 nem Ride Service na porta 8082. Ele chama `http://localhost:8080`, que é o Gateway. O Gateway olha o caminho da requisição e decide para onde enviar. Chamadas para `/accounts` vão para o Account Service. Chamadas para `/rides` vão para o Ride Service. Chamadas para `/ws` vão para o WebSocket do Ride Service.

O arquivo `ApiGatewayApplication.java` é pequeno, porque a maior parte da configuração está no YAML do Gateway. No `api-gateway.yaml` aparecem as rotas. A rota `account-route` usa `uri lb://ACCOUNT-SERVICE` e `predicate Path=/accounts/**`. Isso significa que qualquer requisição cujo caminho comece com `/accounts` deve ser encaminhada para o Account Service. A rota `ride-route` faz o mesmo para `/rides/**`. A rota `ride-ws-route` encaminha WebSocket para o Ride Service.

O Gateway também configura CORS. CORS é uma regra de segurança do navegador que controla quais origens podem chamar a API. Como o frontend roda em `http://localhost:4200` e o backend em `http://localhost:8080`, o navegador considera origens diferentes. Por isso, o Gateway permite a origem do frontend, os métodos HTTP principais e headers.

No nível iniciante, explique que o Gateway é como uma recepção. O usuário chega por uma porta única e a recepção direciona para o setor correto. No nível intermediário, explique que ele roteia caminhos para microserviços usando Eureka. No nível avançado, explique que ele também centraliza preocupações transversais, como CORS, logs, segurança e roteamento, mesmo que nem todas estejam completamente implementadas neste desafio.

Na demonstração, quando você mostrar que o frontend usa `http://localhost:8080`, destaque que esse é o Gateway. Isso ajuda a banca a entender que o navegador não está acoplado aos serviços internos.

Capítulo 10 - Account Service explicado como modulo

O Account Service é o microserviço responsável por contas. Ele permite criar uma conta, listar contas e buscar uma conta por id. No domínio do projeto, uma conta pode ser de cliente ou de motorista. O cliente cria corridas. O motorista aceita corridas. Por isso, o Account Service é consultado pelo Ride Service sempre que precisa validar se um usuário existe ou se uma conta é realmente de motorista.

O arquivo `AccountServiceApplication.java` inicia o serviço. Ele não tem regra de negócio. Isso é proposital. Em uma aplicação bem organizada, a classe principal deve apenas iniciar o Spring. A regra fica em outros pacotes. O pacote `domain` contém a entidade `Account`, o enum `AccountType` e a interface `AccountGateway`. O pacote `application` contém os casos de uso de criar, buscar e listar. O pacote `infrastructure` contém `controller`, `models`, `presenter`, entidade `JPA`, `repository`, `gateway PostgreSQL` e tratamento de erros.

A entidade `Account` representa uma conta no domínio. Ela tem `id`, `nome`, `email`, `tipo` e `data de criação`. O método `newAccount` cria uma conta nova. Ele valida `nome`, `email` e `tipo`, gera um `UUID` e registra a data atual. O método `with` reconstrói uma conta existente, normalmente quando ela vem do banco. Essa diferença é importante. `newAccount` representa criação nova. `with` representa remontar algo que já existe.

O enum `AccountType` limita os tipos aceitos. O sistema não aceita qualquer texto. Ele aceita `CLIENT` ou `DRIVER`. Essa escolha evita erros de digitação e deixa a regra mais clara. A interface `AccountGateway` define o que o domínio precisa em relação à persistência. Ela permite salvar, buscar por `id` e listar. O domínio não precisa saber que a implementação usa `PostgreSQL`.

No nível avançado, o Account Service demonstra uma estrutura limpa e simples. Ele separa domínio, caso de uso e infraestrutura. Isso faz com que a regra de criação de conta possa ser testada sem subir banco, servidor HTTP ou Docker.

Capítulo 11 - Account Service por dentro dos casos de uso

O caso de uso de criação de conta começa em `CreateAccountCommand`. Esse record carrega nome, email e tipo. Ele representa o pedido de criação vindo de fora. O `CreateAccountOutput` carrega o resultado do caso de uso, que nesse caso é o id criado. `CreateAccountUseCase` define o contrato abstrato com o método `execute`. `DefaultCreateAccountUseCase` implementa esse contrato.

Quando `DefaultCreateAccountUseCase` executa, ele cria uma entidade `Account` usando `Account.newAccount`. Isso significa que a validação principal acontece dentro da entidade, e não espalhada pelo caso de uso. Depois, o caso de uso chama `accountGateway.save` e retorna o id salvo. Essa sequência é simples, mas mostra bem a separação de responsabilidade. A entidade valida e representa a regra. O caso de uso coordena a ação. O gateway persiste.

O caso de uso de busca por id segue outra lógica. `DefaultGetAccountByIdUseCase` chama `accountGateway.getById`. Como a busca pode não encontrar nada, o gateway retorna `Optional`. Se não existir conta, o caso de uso lança `NoSuchElementException`. Mais tarde, o `GlobalException` transforma isso em uma resposta HTTP 404. Isso é melhor do que retornar `null`, porque `null` poderia causar erro inesperado.

O caso de uso de listagem chama `accountGateway.getAll` e transforma cada `Account` em `ListAccountsOutput`. Essa transformação evita que a API exponha diretamente a entidade do domínio. Mesmo quando os campos são parecidos, separar output de entidade é uma prática que protege o domínio de mudanças externas.

Para explicar na apresentação, diga que casos de uso representam ações do sistema. Criar conta, buscar conta e listar contas são casos de uso. Eles não conhecem HTTP nem banco diretamente. Eles conhecem comandos, outputs e gateways. Isso permite testar essas ações isoladamente.

Capítulo 12 - Account Service na API e no banco

A API do Account Service é declarada em `AccountAPI.java`. Esse arquivo define os endpoints HTTP. O `POST /accounts` cria conta. O `GET /accounts/{id}` busca uma conta específica. O `GET /accounts` lista todas as contas. A interface usa anotações como `RequestMapping`, `PostMapping`, `GetMapping`, `RequestBody`, `PathVariable` e `Valid`. Essas anotações dizem ao Spring como transformar uma chamada HTTP em uma chamada de método Java.

O `AccountController` implementa `AccountAPI`. Quando chega uma chamada para criar conta, o controller recebe `CreateAccountRequest`, cria `CreateAccountCommand`, executa o caso de uso e devolve `CreateAccountResponse` com status 201 Created. Quando chega uma chamada para buscar por id, o controller chama `GetAccountByIdUseCase` e usa `AccountPresenter` para transformar o output em `AccountResponse`. Quando chega uma chamada para listar, ele chama `ListAccountsUseCase` e transforma a lista de outputs em responses.

Os models representam os formatos JSON da API. `CreateAccountRequest` representa o corpo que vem do frontend. `CreateAccountResponse` representa a resposta depois de criar conta. `AccountResponse` e `ListAccountsResponse` representam os dados que voltam em consultas. Separar models da entidade de domínio evita que detalhes internos vazem para a camada HTTP.

No banco, `AccountEntity` é a classe JPA. Ela tem anotações `Entity` e `Table` para mapear a tabela `tb_accounts`. Cada campo tem `Column` com nome de coluna. O campo `type` usa `Enumerated` com `EnumType.STRING`, o que faz `CLIENT` ou `DRIVER` serem salvos como texto. `AccountRepository` estende `JpaRepository`, então ganha métodos prontos como `save`, `findById` e `findAll`.

`AccountPostgresGateway` implementa `AccountGateway`. Ao salvar, ele converte `Account` para `AccountEntity`. Ao buscar, converte `AccountEntity` para `Account`. Essa conversão parece trabalhosa no começo, mas é importante para manter o domínio separado da persistência. Na apresentação, diga que `AccountEntity` é banco, `Account` é regra de negócio. Elas se parecem, mas não têm a mesma responsabilidade.

Capítulo 13 - Tratamento de erros no Account Service

O tratamento de erros do Account Service fica em `GlobalException.java`. Esse arquivo usa `RestControllerAdvice`, que permite capturar exceções lançadas pelos controllers e casos de uso e transformar em respostas HTTP padronizadas. Sem esse tratamento, cada controller precisaria repetir `try` e `catch`, deixando o código mais poluído.

Quando ocorre erro de validação em campos anotados com validação, o Spring lança `MethodArgumentNotValidException`. O `GlobalException` percorre os erros de campo e monta um mapa com nome do campo e mensagem. A resposta recebe status 400 Bad Request, porque o problema veio nos dados enviados pelo cliente.

Quando o corpo da requisição está malformado ou contém valor inválido para enum, pode ocorrer `HttpMessageNotReadableException`. O sistema responde 400 com uma mensagem dizendo que o body está inválido. Quando a própria regra de negócio lança `IllegalArgumentException`, isso também vira 400, porque representa argumento inválido. Quando a conta não existe e o caso de uso lança `NoSuchElementException`, a resposta vira 404 Not Found. Para `RuntimeException` genérica, a resposta vira 500 Internal Server Error.

`CustomErrorResponse` padroniza o corpo do erro. Em vez de cada erro voltar de um jeito diferente, a API devolve uma estrutura com um mapa de mensagens. Isso facilita o frontend, porque ele pode tentar extrair mensagens desse formato.

Na apresentação, explique que tratar erros globalmente ajuda a manter controllers limpos e respostas previsíveis. Também ajuda o usuário, porque em vez de ver erro técnico ou stack trace, ele recebe uma mensagem mais controlada.

No nível avançado, destaque que a API diferencia erro do cliente e erro do servidor. Dados inválidos viram 400. Recurso inexistente vira 404. Falha inesperada vira 500. Essa diferença é importante para debugar e para criar uma boa experiência no frontend.

Capítulo 14 - Ride Service explicado como modulo principal

O Ride Service e o modulo mais importante do projeto, porque concentra o fluxo de corridas. Ele cria corridas, edita origem e destino, lista corridas, busca por id, consulta status, aceita corrida, cancela corridas antigas por timeout, publica evento Kafka, consome evento e envia notificacao WebSocket.

O arquivo RideServiceApplication.java inicia o servico. Alem de SpringBootApplication, ele tem EnableFeignClients e EnableScheduling. EnableFeignClients habilita o uso de OpenFeign para chamar o Account Service. EnableScheduling habilita jobs agendados, como o job que cancela corridas antigas. Esse arquivo mostra que o Ride Service precisa de comunicacao entre servicos e de tarefas automaticas em segundo plano.

O pacote domain contem a entidade Ride, o enum RideStatus e contratos como RideGateway, AccountClient, SentEventService, RideStatusCache e DriverNotifier. O pacote application contem os casos de uso de criar, aceitar, atualizar, listar, buscar, consultar status, mudar status e aplicar timeout. O pacote infrastructure contem API REST, persistencia JPA, Redis, Kafka, WebSocket, Feign, job e configuracao de beans.

Quando voce explicar esse modulo, destaque que ele orquestra varias tecnologias, mas a regra principal continua organizada. O caso de uso nao escreve SQL, nao manipula socket diretamente e nao chama KafkaTemplate diretamente. Ele chama contratos. As implementacoes ficam na infraestrutura.

No nivel iniciante, diga que o Ride Service e o servico das corridas. No nivel intermediario, diga que ele coordena conta, banco, fila, cache e tempo real. No nivel avancado, diga que ele usa arquitetura em camadas e inversao de dependencia para manter a regra de negocio independente dos detalhes externos.

Capítulo 15 - A entidade Ride explicada com cuidado

Ride.java representa uma corrida dentro do domínio. Ela tem id, userId, driverId, startAddress, destinationAddress, status, createdAt e updatedAt. O id identifica a corrida. O userId identifica o cliente que criou. O driverId identifica o motorista que aceitou, mas pode ser nulo enquanto a corrida esta disponível. startAddress e destinationAddress guardam origem e destino. status mostra o estado da corrida. createdAt e updatedAt registram datas.

O método newRide cria uma corrida nova. Ele valida userId, origem e destino. Depois gera UUID, pega o horário atual com Instant.now e cria uma corrida com driverId nulo e status CREATED. Essa regra é muito importante. Toda corrida nova nasce sem motorista e aguardando aceite. Isso representa o estado inicial do fluxo.

O método with reconstrói uma corrida existente. Ele é usado quando o sistema lê dados do banco e precisa transformar RideEntity em Ride. Ele não gera id novo nem data nova, porque esta reconstruindo um objeto que já existia. Essa distinção ajuda a evitar confusão entre criação e leitura.

O método assignDriver representa o aceite da corrida. Ele valida se o driverId não é nulo nem vazio. Depois atribui o motorista, muda o status para IN_PROGRESS e atualiza updatedAt. Isso mostra que aceitar uma corrida não é apenas preencher um campo. É uma transição de estado.

O método updateRoute permite alterar origem e destino. Antes de alterar, ele verifica se a corrida esta COMPLETED ou CANCELLED. Se estiver, lança erro, porque uma corrida finalizada ou cancelada não deve ser editada. Depois valida os novos endereços, altera os campos e atualiza updatedAt.

O método changeStatus muda o status e atualiza a data. Ele é usado em fluxos como timeout. Os métodos validate e validateRoute garantem que dados obrigatórios não sejam nulos nem vazios. No nível avançado, diga que Ride é uma entidade rica, porque contém comportamento e regras. Ela não é apenas um conjunto de getters.

Capítulo 16 - O ciclo de vida da corrida

O ciclo de vida da corrida é controlado pelo enum `RideStatus`. Uma corrida com status `CREATED` foi criada e ainda está aguardando motorista. Uma corrida com status `IN_PROGRESS` foi aceita e está em andamento. Uma corrida com status `COMPLETED` foi concluída. Uma corrida com status `CANCELLED` foi cancelada.

Mesmo que o projeto não tenha uma tela completa para concluir corrida, o status `COMPLETED` existe para representar um estado final possível. Isso mostra que o modelo foi pensado como um fluxo de corrida, não apenas como uma tabela simples. O status `CANCELLED` aparece no timeout, quando uma corrida fica tempo demais sem ser aceita.

Quando o cliente cria corrida, o status inicial é `CREATED`. Quando o motorista aceita, `assignDriver` muda para `IN_PROGRESS`. Quando o job de timeout encontra uma corrida antiga ainda em `CREATED`, ele muda para `CANCELLED`. O método `updateRoute` bloqueia edição quando a corrida está `COMPLETED` ou `CANCELLED`.

Na apresentação, explique que status é uma forma de controlar regras. Sem status, o sistema não saberia se uma corrida pode ser aceita, editada ou cancelada. O status também ajuda o frontend a mostrar mensagens diferentes para cliente e motorista. Para o cliente, `CREATED` aparece como "Aguardando motorista". Para o motorista, `CREATED` aparece como "Disponível". A regra é a mesma, mas a linguagem muda conforme a perspectiva.

No nível avançado, você pode dizer que o sistema poderia evoluir para uma máquina de estados mais formal. Por enquanto, as transições principais estão nos métodos da entidade e nos casos de uso. Isso é suficiente para o desafio e deixa o comportamento claro.

Capítulo 17 - Contratos do dominio no Ride Service

O Ride Service define varias interfaces no dominio. Essas interfaces existem porque a regra de negocio precisa de algumas capacidades, mas nao deve depender diretamente da tecnologia que implementa essas capacidades. RideGateway e o contrato de persistencia. Ele permite salvar, buscar por id, listar todas e buscar corridas criadas antes de uma data. A implementacao concreta e RidePostgresGateway.

AccountClient e o contrato para consultar contas. O Ride Service precisa saber se um cliente ou motorista existe, mas nao deve conhecer detalhes internos do Account Service. A implementacao concreta usa OpenFeign. AccountData e um record simples com os dados de conta que o Ride Service precisa: id, nome e tipo.

SentEventService e o contrato para enviar evento. Na implementacao atual, o evento vai para Kafka. Mas o caso de uso de criacao nao precisa conhecer KafkaTemplate. Ele sabe apenas que precisa enviar um evento de corrida criada. DriverNotifier e o contrato para notificar motoristas. A implementacao atual usa WebSocket, mas a regra de negocio depende apenas do contrato.

RideStatusCache e o contrato para cache de status. A implementacao atual usa Redis. RideStatusData representa o dado retornado pelo cache. RideNotification representa a mensagem enviada para motoristas, contendo dados da corrida e status.

Essa organizacao permite explicar um conceito avancado de forma simples. A regra de negocio diz o que precisa acontecer. A infraestrutura decide como isso acontece. O dominio pede "salve a corrida", e o PostgreSQL executa. O dominio pede "envie evento", e Kafka executa. O dominio pede "notifique motorista", e WebSocket executa. O dominio pede "guarde status rapido", e Redis executa.

Capítulo 18 - Criacao de corrida no backend

O fluxo de criacao de corrida comeca quando o controller recebe um `CreateRideRequest`. Esse request vem do frontend com `userId`, `startAddress` e `destinationAddress`. O `RideController` transforma esse request em `CreateRideCommand`. O command e passado para `CreateRideUseCase`, cuja implementacao concreta e `DefaultCreateRideUseCase`.

Dentro de `DefaultCreateRideUseCase`, o primeiro passo e criar a entidade `Ride` usando `Ride.newRide`. Isso valida os dados e define o status inicial como `CREATED`. Depois, o caso de uso consulta `accountClient.getByld` usando o `userId`. Essa consulta confirma se o cliente existe. Se o cliente nao existir, o caso de uso lanca `IllegalArgumentException`.

Depois de validar a conta, o caso de uso salva a corrida pelo `RideGateway`. A implementacao concreta converte `Ride` em `RideEntity` e salva no PostgreSQL. Em seguida, o caso de uso publica um evento usando `sentEventService.sentEvent` com `RideCreatedMessage.from(aRide)`. Essa mensagem contem os dados necessarios para avisar que a corrida foi criada.

O caso de uso envolve a publicacao do evento em try e catch. Se Kafka falhar, o erro e registrado no log e relancado. Isso significa que a API nao esconde a falha de publicacao. Para um projeto mais avancado de producao, seria possivel usar outbox pattern para garantir consistencia entre banco e evento, mas para o desafio a estrategia atual deixa a falha visivel.

Na apresentacao, voce pode dizer que a criacao da corrida mostra o projeto inteiro comecando a se movimentar. Ela valida dados, consulta outro microservico, salva no banco e inicia uma comunicacao assincrona via Kafka. Esse fluxo e um dos melhores para demonstrar a arquitetura.

Capítulo 19 - Aceite de corrida no backend

O aceite de corrida acontece quando o motorista clica em aceitar no frontend. A chamada chega em POST `/rides/{id}/accept`. O `RideController` recebe o id da URL e o `driverId` do corpo da requisicao. Ele cria `AcceptRideCommand` e chama `AcceptRideUseCase`. A implementacao concreta e `DefaultAcceptRideUseCase`.

O primeiro passo do caso de uso e consultar o `Account Service` pelo `driverId`. Se a conta nao existe, o sistema lanca erro. Se a conta existe, o caso de uso verifica se o tipo da conta e `DRIVER`. Isso impede que um cliente aceite uma corrida como se fosse motorista. Essa regra e importante porque o frontend sozinho nao deve ser a unica protecao.

Depois, o caso de uso busca a corrida pelo `RideGateway`. Se a corrida nao existe, lanca `NoSuchElementException`. Se existe, verifica se o status e `CREATED`. Caso nao seja, significa que a corrida ja foi aceita, cancelada ou finalizada. Nesse caso, o sistema lanca `IllegalStateException`. Essa regra evita aceitar uma corrida que nao esta mais disponivel.

Quando tudo esta valido, o caso de uso chama `aRide.assignDriver`. Esse metodo atribui o motorista, muda o status para `IN_PROGRESS` e atualiza `updatedAt`. Depois a corrida e salva no banco. Em seguida, o sistema tenta gravar o status no Redis. Se o Redis falhar, o erro e logado, mas a corrida continua salva no PostgreSQL. Depois o sistema tenta notificar via `WebSocket`. Se a notificacao falhar, o erro tambem e logado.

Essa decisao mostra que o PostgreSQL e a fonte principal de verdade. Redis e `WebSocket` sao importantes, mas complementares. Na apresentacao, explique que o aceite prioriza a consistencia do dado principal. A corrida aceita fica salva no banco, mesmo que uma notificacao temporaria falhe.

Capítulo 20 - Edicao, consulta de status e timeout de corridas

O caso de uso de edicao permite alterar origem e destino de uma corrida. Ele recebe `UpdateRideCommand` com `rideId`, `userId`, `startAddress` e `destinationAddress`. Primeiro valida se `userId` foi informado. Depois busca a corrida. Em seguida, verifica se o `userId` enviado e igual ao `userId` da corrida. Isso impede que um cliente edite corrida de outro cliente. Depois chama `updateRoute` na entidade `Ride`, salva a corrida e notifica motoristas sobre a atualizacao.

A consulta de status tem uma ideia importante: tentar cache primeiro. `DefaultGetRideStatusUseCase` chama `rideStatusCache.get`. Se encontrar dado no Redis, retorna status com `source` igual a `redis`. Se nao encontrar, busca a corrida no PostgreSQL e retorna `source` igual a `database`. Esse campo `source` e interessante porque mostra de onde veio a resposta. Na demonstracao, ele pode ajudar a explicar o papel do Redis.

O timeout e uma rotina automatica. `RideTimeoutJob` executa em intervalo configurado por `ride.timeout.check-interval-ms`. Ele calcula uma data limite subtraindo `ride.timeout.seconds` do horario atual. Depois chama `TimeoutRidesUseCase` com essa data. O caso de uso busca corridas com status `CREATED` criadas antes do limite. Para cada corrida expirada, muda o status para `CANCELLED`, salva no banco, atualiza Redis e notifica via `WebSocket`.

O repository usa o metodo `findTop50ByStatusAndCreatedAtBefore`. O Spring Data JPA interpreta esse nome e cria a consulta automaticamente. Buscar no maximo cinquenta por execucao evita que o job tente processar um volume ilimitado de uma vez. Para o desafio, isso e uma boa forma de mostrar preocupacao com controle de processamento.

Na apresentacao, diga que esses fluxos mostram regras alem do cadastro basico. O sistema controla propriedade da edicao, otimiza consulta de status com cache e executa cancelamento automatico de corridas antigas.

Capítulo 21 - Persistencia de corridas com PostgreSQL e JPA

O PostgreSQL é usado como banco persistente. Persistente significa que os dados são armazenados de forma durável. No projeto, contas ficam no banco `account_db` e corridas ficam no banco `ride_db`. O Docker Compose cria o container PostgreSQL e o arquivo SQL de inicialização cria o banco `ride_db`.

No Ride Service, `RideEntity` é a classe JPA que representa a tabela `tb_rides`. Ela possui anotações `Entity` e `Table`. Cada campo da classe é mapeado para uma coluna. O `id` vira `ride_id`. O `userId` vira `ride_user_id`. O `driverId` vira `ride_driver_id`. A origem vira `ride_start_address`. O destino vira `ride_destination_address`. O status vira `ride_status`. As datas viram `ride_created_at` e `ride_updated_at`.

`RideEntity` tem dois métodos muito importantes. O método `from` recebe uma entidade de domínio `Ride` e cria uma entidade JPA. O método `toAggregate` recebe uma entidade JPA e reconstrói uma entidade de domínio `Ride`. Essa conversão separa banco e regra de negócio. A entidade de domínio não tem anotações JPA. Ela não precisa saber nome de tabela nem nome de coluna.

`RideRepository` estende `JpaRepository`. Isso significa que o Spring Data fornece automaticamente métodos como `save`, `findById` e `findAll`. Além disso, o `repository` declara `findTop50ByStatusAndCreatedAtBefore`, que é usado no `timeout`. Esse método mostra como o Spring Data consegue criar consultas a partir do nome do método.

`RidePostgresGateway` implementa `RideGateway`. Ele é o adaptador que liga domínio e banco. Quando o caso de uso pede para salvar uma corrida, ele não fala com `JpaRepository` diretamente. Ele fala com `RideGateway`. A implementação concreta converte e salva. Esse desenho ajuda testes e manutenção.

Capítulo 22 - Kafka explicado no fluxo do projeto

Kafka é usado como broker de mensagens. Um broker de mensagens permite que uma parte do sistema publique um acontecimento e outra parte consuma esse acontecimento. No projeto, quando uma corrida é criada, o sistema publica uma mensagem no tópico `ride-topic`. Depois, um consumidor recebe essa mensagem e envia notificação para motoristas via `WebSocket`.

`KafkaRideConfig` cria o tópico `ride-topic`. `RideProducer` é a classe que publica mensagens. Ela usa `KafkaTemplate` para enviar `RideCreatedMessage`. `SentEventServiceImpl` implementa o contrato `SentEventService` e chama `RideProducer`. Essa divisão permite que o caso de uso dependa de `SentEventService`, não de `KafkaTemplate` diretamente.

`RideConsumer` é a classe que consome mensagens. O método `consume` tem a anotação `KafkaListener` com `topics` igual a `ride-topic`. Quando uma mensagem chega, o Spring chama esse método automaticamente. O consumer transforma a mensagem em `RideNotification` e chama `DriverNotifier`. A implementação atual de `DriverNotifier` envia via `WebSocket`.

No nível iniciante, diga que Kafka funciona como uma fila de avisos. No nível intermediário, explique que ele desacopla criação de corrida e notificação. No nível avançado, diga que esse desenho facilita evolução. No futuro, outros consumidores poderiam ouvir o mesmo tópico para gerar métricas, auditoria ou histórico, sem mudar o caso de uso de criação.

Também é importante explicar por que não chamar `WebSocket` diretamente no caso de uso de criação. Seria possível em um projeto pequeno, mas Kafka torna o fluxo mais flexível. A criação da corrida publica um evento, e os interessados reagem ao evento. Isso é uma prática comum em arquiteturas orientadas a eventos.

Capítulo 23 - Redis explicado no fluxo do projeto

Redis é usado como cache de status. Cache é uma camada de armazenamento rápido, geralmente temporária, usada para acelerar leituras. No projeto, o Redis guarda o status da corrida e o id do motorista em uma chave baseada no id da corrida. A chave usa o prefixo `ride:status:` seguido do id.

`RideStatusCache` é o contrato do domínio. `RideStatusRedisCache` é a implementação com Redis. Quando uma corrida é aceita, `DefaultAcceptRideUseCase` chama `rideStatusCache.put`. Quando uma corrida expira por timeout, `DefaultTimeoutRidesUseCase` também atualiza o cache. O Redis guarda status e `driverId` em formato de hash. Depois aplica um TTL de vinte e quatro horas. TTL significa tempo de vida. Depois desse tempo, a chave pode expirar.

Quando alguém consulta o endpoint de status, `DefaultGetRideStatusUseCase` tenta ler Redis primeiro. Se encontrar, responde rapidamente e informa `source redis`. Se não encontrar, busca no PostgreSQL e informa `source database`. Isso mostra uma estratégia conhecida como `cache-aside`. A aplicação consulta cache, mas o banco continua sendo a fonte persistente.

Na apresentação, deixe claro que Redis não substitui PostgreSQL. Ele complementa. Se Redis cair ou perder uma chave, o status principal ainda está salvo no banco. Isso é importante para evitar uma interpretação errada de que o sistema depende somente do cache.

No nível avançado, você pode dizer que esse cache melhora tempo de resposta para consultas frequentes de status. Também mostra uma preocupação com escalabilidade, mesmo que o projeto seja um desafio acadêmico.

Capítulo 24 - WebSocket e STOMP explicados no projeto

WebSocket é a tecnologia usada para notificações em tempo real. Em HTTP comum, o navegador faz uma pergunta e o servidor responde. Depois a conexão acaba. Em WebSocket, existe uma conexão aberta, e o servidor consegue enviar mensagens para o navegador quando algo acontece. Isso é ideal para avisar motoristas sobre novas corridas.

WebSocketConfig habilita WebSocket com STOMP. O endpoint registrado é /ws. O broker simples usa o prefixo /topic. Isso significa que o backend pode publicar em tópicos como /topic/rides, e clientes conectados podem assinar esse tópico. STOMP é um protocolo de mensagens sobre WebSocket que organiza a comunicação em destinos, como tópicos.

WebSocketDriverNotifier implementa DriverNotifier. Ele usa SimpMessagingTemplate para enviar RideNotification ao tópico /topic/rides. O domínio conhece apenas DriverNotifier. Ele não sabe que por baixo existe STOMP, SimpMessagingTemplate ou WebSocket. Isso mostra novamente a separação entre regra e infraestrutura.

No frontend, RideNotificationService usa RxStomp. Ele configura brokerURL com environment.wsUrl, define reconexão e heartbeat, ativa a conexão e assina /topic/rides. Quando recebe uma mensagem, transforma o corpo JSON em RideNotificationDto. A tela do motorista usa esse service para atualizar a lista de corridas disponíveis.

Na apresentação, explique que WebSocket evita polling. Polling seria fazer o frontend perguntar a cada poucos segundos se existe corrida nova. Com WebSocket, o servidor avisa quando a corrida aparece. Isso reduz atraso e evita requisições repetidas desnecessárias.

Capítulo 25 - OpenFeign explicado no projeto

OpenFeign é usado para o Ride Service consultar o Account Service. O Ride Service precisa saber se o cliente existe quando cria uma corrida e se o motorista existe e é do tipo DRIVER quando aceita uma corrida. Como contas pertencem ao Account Service, o Ride Service faz uma chamada HTTP entre microserviços.

AccountFeignClient declara essa chamada. Ele tem a anotação FeignClient com name igual a account-service. O método getAccountById tem GetMapping para /accounts/{id}. Com isso, o código Java parece uma chamada de método comum, mas por baixo o Feign faz uma requisição HTTP para o serviço correto.

AccountClientImpl implementa o contrato AccountClient do domínio. Ele usa AccountFeignClient, recebe AccountFeignResponse e transforma em AccountData. Se o Account Service responder 404, o Feign lança FeignException.NotFound. O código captura essa exceção e retorna Optional.empty. Isso permite que o caso de uso trate ausência de conta de forma clara.

O ponto mais importante é que o domínio não depende de Feign. Ele depende de AccountClient. Feign fica na infraestrutura. Isso facilita testes, porque nos testes unitários dos casos de uso é possível mockar AccountClient sem fazer chamada HTTP real.

Na apresentação, diga que OpenFeign simplifica comunicação entre microserviços, mas foi encapsulado por uma interface do domínio para manter baixo acoplamento. Essa frase mostra entendimento de ferramenta e de arquitetura.

Capítulo 26 - Docker e Docker Compose explicados

Docker empacota aplicações em containers. Um container é um ambiente isolado com tudo que uma aplicação precisa para rodar. Docker Compose permite subir vários containers juntos com um arquivo de configuração. No projeto, o `docker-compose.yml` sobe PostgreSQL, Kafka, Redis, Config Server, Eureka Server, Account Service, Ride Service, API Gateway e frontend.

O container postgres usa a imagem `postgres:16-alpine`. Ele expõe a porta 5433 na máquina e usa 5432 internamente. Define usuário, senha e banco `account_db`. Também monta um volume chamado `postgres_data` para preservar dados e monta a pasta `docker/init` para executar scripts de inicialização, como a criação de `ride_db`.

O Kafka usa uma imagem da Confluent e roda em modo KRaft. O Redis usa `redis:7-alpine`. Os serviços Java são construídos a partir dos Dockerfiles de cada módulo. O frontend é construído a partir do Dockerfile do Angular e servido por Nginx. O Gateway expõe a porta 8080, o Eureka expõe 8761 e o frontend expõe 4200.

O Compose usa healthchecks para verificar se os serviços estão saudáveis. Por exemplo, o PostgreSQL usa `pg_isready`. O Redis usa `redis-cli ping`. Os serviços Java usam endpoint de `actuator health`. O `depends_on` com `condition service_healthy` faz um serviço esperar outro estar pronto antes de iniciar. Isso reduz problemas de subida em ordem errada.

Na apresentação, explique que Docker Compose torna a demonstração reproduzível. Em vez de instalar tudo manualmente, o avaliador ou desenvolvedor consegue subir o ambiente com `docker compose up --build`. No nível avançado, destaque que dentro da rede Docker os serviços se comunicam pelo nome do container, como `postgres`, `kafka` e `redis`, e não por `localhost`.

Capítulo 27 - Angular explicado no projeto

Angular é o framework usado no frontend. Ele organiza a interface em componentes, rotas, services, formulários e estado reativo. O projeto usa Angular com componentes standalone, o que significa que cada componente declara os imports que precisa diretamente, sem depender de um módulo tradicional.

O arquivo `main.ts` é o ponto de entrada do frontend. Ele chama `bootstrapApplication` passando o componente `App` e a configuração `appConfig`. Isso inicia a aplicação Angular no navegador. O `app.config.ts` registra providers importantes. Ele define `API_BASE_URL` com `environment.apiUrl`, configura `HttpClient` com interceptor de erro, registra as rotas, configura `PrimeNG` e disponibiliza `MessageService` para mensagens toast.

O `app.routes.ts` define as rotas da aplicação. A rota `/login` carrega a tela de login. A rota `/client/rides` carrega a tela do cliente, protegida por `sessionGuard` e `role guard` de `CLIENT`. A rota `/driver/rides/available` carrega a tela do motorista, protegida por `sessionGuard` e `role guard` de `DRIVER`. A rota `/forbidden` mostra acesso negado. A rota vazia usa um dashboard para redirecionar conforme o perfil.

O componente `App` monta a moldura geral da aplicação. Ele mostra o título, a navegação, a sidebar com a conta selecionada, o botão sair e o `router-outlet`. O `router-outlet` é o local onde a tela da rota atual aparece. O `App` também usa `p-toast` para mensagens globais.

Na apresentação, diga que Angular foi escolhido porque facilita criar uma SPA organizada. SPA significa *single page application*, uma aplicação que troca telas sem recarregar toda a página. O projeto usa recursos modernos como `signals` e `computed` para atualizar a interface de forma reativa.

Capítulo 28 - TypeScript, DTOs e services do frontend

TypeScript é a linguagem usada no Angular. Ele é uma evolução do JavaScript com tipos. Tipos ajudam a saber quais campos existem em um objeto e reduzem erros. No projeto, `api-dtos.ts` define os formatos dos dados que circulam entre frontend e backend. `AccountDto` representa uma conta. `RideDto` representa uma corrida. `CreateRideRequestDto` representa o corpo usado para criar corrida. `RideNotificationDto` representa a mensagem recebida por WebSocket.

O arquivo `api-routes.ts` centraliza a montagem das URLs. Em vez de cada componente escrever strings de rota manualmente, o projeto usa funções para montar endpoints de contas, corridas, status e aceite. Isso reduz repetição e diminui risco de erro de digitação.

`AccountService` usa `HttpClient` para chamar endpoints de conta. Ele tem métodos para criar conta, listar contas e buscar conta por id. `RideService` faz o mesmo para corridas. Ele cria, edita, lista, busca por id, consulta status e aceita corrida. Esses services retornam `Observables`, que representam respostas assíncronas. O componente se inscreve no `Observable` usando `subscribe` para reagir quando a resposta chega.

O token `API_BASE_URL` permite injetar a URL base da API. Isso evita espalhar `environment.apiUrl` por toda a aplicação. Se a URL mudar, a configuração fica concentrada.

Na apresentação, explique que DTOs funcionam como moldes dos dados. Eles ajudam o frontend a saber que uma corrida tem `id`, `userId`, `driverId`, `origem`, `destino`, `status` e `datas`. Também ajudam a tratar casos como `driverId` nulo quando a corrida ainda não tem motorista.

Capítulo 29 - Sessao, guards e erros no frontend

O SessionService controla a conta selecionada. Ele usa localStorage para manter a conta mesmo se a pagina for recarregada. Tambem usa signal para manter o estado reativo dentro da aplicacao. O metodo login salva a conta no localStorage e atualiza o signal. O metodo logout remove a conta e limpa o signal. O computed type retorna o tipo da conta atual, que pode ser CLIENT, DRIVER ou null.

E importante dizer que isso nao e autenticacao real. Nao existe senha, token JWT nem validacao de permissao no backend. Para o desafio, a sessao serve para escolher um perfil e demonstrar o fluxo. Em uma evolucao real, seria necessario implementar login seguro, tokens e autorizacao no backend.

O sessionGuard impede acessar rotas internas sem conta selecionada. Se nao houver conta, ele redireciona para /login. O role guard verifica o tipo da conta. Se uma rota exige CLIENT e o usuario e DRIVER, ele redireciona para /forbidden. Isso melhora a experiencia e evita que um usuario navegue para tela errada pelo frontend.

O errorHandler captura erros HTTP. Quando uma chamada falha, ele usa getMessageForHttpError para transformar status e corpo de erro em mensagem amigavel. Depois usa MessageService para mostrar um toast. O arquivo error-messages.ts define mensagens para 400, 404, 409 e 500. Se o backend envia mensagens no corpo errors, o frontend tenta mostrar essas mensagens.

Na apresentacao, explique que o frontend tem tratamento centralizado de erro. Isso evita repetir o mesmo tratamento em cada componente. Tambem deixa a experiencia mais clara para o usuario.

Capítulo 30 - Tela de login explicada

A tela de login fica em `login.component.ts`, `login.component.html` e `login.component.css`. Ela permite escolher uma conta existente ou criar uma nova conta. O usuário pode alternar entre cliente e motorista. A lista exibida muda conforme o tipo selecionado.

No TypeScript, o componente injeta `AccountService`, `SessionService`, `Router`, `DestroyRef` e `FormBuilder`. `AccountService` busca e cria contas. `SessionService` salva a conta selecionada. `Router` navega depois da entrada. `DestroyRef` ajuda a encerrar assinaturas quando o componente é destruído. `FormBuilder` cria o formulário reativo de cadastro.

O componente usa `signals` para controlar estado. `accounts` guarda a lista de contas. `selectedType` guarda `CLIENT` ou `DRIVER`. `loading` indica carregamento. `createDialogVisible` controla se o modal está aberto. `creating` indica se o cadastro está sendo enviado. O `computed filteredAccounts` filtra a lista de contas pelo tipo selecionado.

O formulário de criação tem `name`, `email` e `type`. O nome é obrigatório. O email é obrigatório e precisa ter formato válido. O `type` é obrigatório. Quando o usuário envia o formulário, o componente valida os campos, chama `createAccount`, fecha o dialog e entra automaticamente com a conta criada.

No HTML, a tela mostra uma área visual do projeto, os botões de perfil, a lista de contas, o botão de atualizar, o botão de criar conta e o dialog de cadastro. Na apresentação, explique que essa tela substitui um login real para facilitar a demonstração dos perfis. Ela é uma seleção de contexto para o desafio.

Capítulo 31 - Tela do cliente explicada

A tela do cliente fica em `client-rides.component.ts`, `client-rides.component.html` e `client-rides.component.css`. Ela permite ao cliente ver suas corridas, criar uma nova corrida e editar origem e destino quando a corrida ainda pode ser editada.

O componente injeta `RideService`, `SessionService`, `MessageService`, `DestroyRef` e `FormBuilder`. `RideService` chama a API de corridas. `SessionService` informa a conta atual. `MessageService` mostra toasts de sucesso ou erro. `FormBuilder` cria formulários de criação e edição.

O estado principal é `rides`, que guarda a lista de corridas carregadas. O `computed myRides` filtra apenas as corridas cujo `userId` é igual ao `id` da conta atual. Isso faz a tela mostrar as corridas do cliente selecionado. `loading` controla carregamento. `createDialogVisible` controla o diálogo de nova corrida. `creating` controla envio. `editingRideId` informa qual corrida está em edição. `savingRideId` informa qual corrida está sendo salva.

Quando o cliente cria uma corrida, o componente pega origem e destino do formulário, pega o `id` da conta atual e chama `rideService.createRide`. Depois que a API responde, ele fecha o diálogo, mostra mensagem de sucesso e recarrega a lista. No backend, esse simples clique inicia um fluxo grande: valida cliente, salva corrida, publica Kafka e notifica motorista.

Quando o cliente edita uma corrida, o componente verifica se a corrida pode ser editada. O método `canEditRide` retorna falso se o status for `COMPLETED` ou `CANCELLED`. Ao salvar, o componente chama `updateRide` e atualiza a lista local com a corrida retornada pela API. Na apresentação, destaque que o frontend também protege a interface, mas a regra principal de permissão e status fica no backend.

Capítulo 32 - Tela do motorista explicada

A tela do motorista fica em `driver-available-rides.component.ts`, `driver-available-rides.component.html` e `driver-available-rides.component.css`. Ela mostra corridas disponiveis e permite aceitar uma corrida.

O componente injeta `RideService`, `RideNotificationService`, `SessionService`, `MessageService` e `DestroyRef`. `RideService` lista e aceita corridas. `RideNotificationService` conecta no `WebSocket` e recebe mensagens em tempo real. `SessionService` informa o motorista atual. `MessageService` mostra mensagens. `DestroyRef` encerra assinaturas quando o componente sai da tela.

O estado `rides` guarda as corridas conhecidas pela tela. O `computed availableRides` filtra apenas corridas com status `CREATED` e sem `driverId`. Isso significa que a tela mostra apenas corridas realmente disponiveis para aceite. `acceptingRideId` indica qual corrida esta sendo aceita, para bloquear cliques repetidos e mostrar loading no botao correto.

No constructor, o componente chama `loadRides` e `watchRideNotifications`. O `loadRides` busca o estado atual pela API. O `watchRideNotifications` assina o topico de `WebSocket`. Isso e importante porque a tela combina duas estrategias. Primeiro carrega o que ja existe. Depois continua recebendo novidades em tempo real.

Quando chega uma notificacao, `handleNotification` decide o que fazer. Se a notificacao nao esta em `CREATED` ou ja tem motorista, a corrida e removida da lista. Isso acontece, por exemplo, quando alguem aceita a corrida. Se a notificacao representa uma corrida disponivel, ela e convertida em `RideDto` e adicionada no topo da lista. O usuario tambem recebe um toast dizendo que ha nova corrida disponivel.

Na apresentacao, essa tela e ideal para demonstrar `WebSocket`. Abra a tela do motorista, depois crie uma corrida como cliente. A corrida deve aparecer para o motorista sem precisar atualizar manualmente.

Capítulo 33 - Componentes compartilhados e helpers

O projeto tem componentes e helpers compartilhados para evitar repetição.

`PageHeaderComponent` é um componente de cabeçalho usado nas telas principais. Ele recebe título e subtítulo e permite inserir ações, como botões de atualizar ou nova corrida. Isso evita copiar a mesma estrutura de cabeçalho em várias telas.

`RideStatusTagComponent` mostra o status da corrida como uma tag visual do PrimeNG. Ele recebe o status e a perspectiva, que pode ser `client` ou `driver`. A perspectiva importa porque a mesma informação pode ser exibida com palavras diferentes. Para o cliente, `CREATED` significa "Aguardando motorista". Para o motorista, `CREATED` significa "Disponível". O componente usa `computed` para recalcular label e severidade quando as entradas mudam.

O arquivo `ride-view.helpers.ts` centraliza funções visuais. Ele define labels de status para cliente e motorista, severidades de tag, função para encurtar id, função para mostrar motorista e função de `trackBy` para tabelas. Essa centralização evita que cada tela implemente sua própria versão da mesma regra visual.

Os arquivos de estilo, como `styles.css`, `colors.css` e CSS de cada componente, definem a aparência da aplicação. O CSS global cuida da base visual. O CSS de cada componente cuida dos detalhes da tela correspondente. Essa organização ajuda a manter o visual consistente sem misturar tudo em um arquivo único.

Na apresentação, explique que componentes compartilhados e helpers reduzem duplicação. Isso torna o código mais fácil de manter. Se a label de um status mudar, por exemplo, a alteração pode ser feita no helper central, e não em várias telas.

Capitulo 34 - PrimeNG, RxJS e experiencia do usuario

PrimeNG e a biblioteca de componentes visuais usada no frontend. Ela fornece botoes, dialogos, tabelas, tags, inputs, selects e toasts. Usar uma biblioteca assim acelera o desenvolvimento e deixa a interface mais consistente. No projeto, PrimeNG aparece em imports como Button, Dialog, InputText, Select, TableModule, Tag e Toast.

RxJS e usado porque chamadas HTTP e mensagens WebSocket sao assincronas. Quando o frontend chama uma API, a resposta nao chega imediatamente. O metodo retorna um Observable. O componente usa subscribe para executar uma acao quando a resposta chega. O operador finalize e usado para desligar loading depois que a chamada termina, com sucesso ou erro. O takeUntilDestroyed evita que assinaturas continuem vivas depois que o componente for destruido.

A experiencia do usuario foi pensada com estados de carregamento, mensagens de sucesso, mensagens de erro e bloqueio de botoes durante operacoes. Quando uma corrida esta sendo aceita, o botao mostra loading. Quando uma corrida e criada, aparece toast de sucesso. Quando a API retorna erro, o interceptor mostra uma mensagem amigavel.

Na apresentacao, explique que frontend nao e apenas desenhar tela. Ele tambem precisa controlar estado, chamadas assincronas, validacao, permissao visual e resposta ao usuario. Angular, RxJS e PrimeNG trabalham juntos para entregar essa experiencia.

No nivel avancado, destaque que a tela do motorista e reativa em dois sentidos. Ela reage a chamadas HTTP iniciais e tambem reage a eventos WebSocket que chegam depois. Isso torna a interface mais proxima de uma aplicacao em tempo real.

Capítulo 35 - Testes do backend explicados

Os testes do backend usam JUnit, Mockito, Testcontainers e JaCoCo. JUnit organiza e executa os testes. Mockito cria objetos falsos, chamados mocks, para isolar uma unidade de código. Testcontainers sobe um PostgreSQL real em Docker para testes de integração. JaCoCo mede cobertura de testes.

Os testes de entidade verificam regras internas. AccountTest valida se a entidade Account impede dados inválidos. RideTest valida criação de corrida, atribuição de motorista, edição de rota e regras de status. Esses testes são importantes porque as entidades concentram regras fundamentais.

Os testes de casos de uso verificam a aplicação sem depender de banco real. DefaultCreateAccountUseCaseTest, DefaultGetAccountByIdUseCaseTest e DefaultListAccountsUseCaseTest testam o Account Service. No Ride Service, existem testes para criar corrida, aceitar corrida, atualizar corrida, buscar por id, listar, consultar status e executar timeout. Como usam mocks, esses testes são rápidos e focados na regra.

Os testes de integração, como AccountPostgresGatewayIT e RidePostgresGatewayIT, validam a persistência com PostgreSQL real. Isso é melhor do que usar um banco em memória quando o objetivo é verificar comportamento real de banco, mapeamento JPA e queries. O Testcontainers cria um banco descartável para o teste e remove depois.

O JaCoCo exige cobertura mínima em domínio e aplicação. A infraestrutura fica fora do gate principal porque controllers, adapters e configurações podem ser testados por integração e E2E. Na apresentação, diga que a estratégia prioriza regras de negócio com testes unitários e valida pontos de integração com banco real.

Capítulo 36 - Testes do frontend, Postman e CI

No frontend, os testes unitarios usam Jasmine e Karma. Jasmine define os testes e expectativas. Karma executa os testes no navegador. Os testes verificam componentes, helpers e interceptadores. Por exemplo, o teste do `RideStatusTagComponent` valida se o status aparece com o texto correto. O teste do `ClientRidesComponent` valida filtro, criacao, edicao e regras de formulario. O teste do `errorInterceptor` valida se mensagens de erro sao mostradas corretamente.

Cypress e usado para teste E2E de smoke. Smoke test e um teste rapido que verifica se o fluxo basico funciona. O arquivo `smoke.cy.ts` verifica se visitante sem sessao e redirecionado para login, se a tela de login aparece, se o filtro por perfil funciona e se entrar como cliente leva para a tela de corridas. O teste usa `cy.intercept` para simular respostas da API, entao pode rodar sem backend de pe.

Postman e usado para demonstrar a API manualmente. A collection em postman permite criar cliente, criar motorista, criar corrida, aceitar corrida e consultar status. Isso e util para mostrar que o backend funciona independentemente do frontend.

O GitHub Actions executa CI. CI significa integracao continua. A cada push ou pull request na branch main, o pipeline roda testes e build. O job do backend configura JDK 17 e executa Maven verify. O job do frontend configura Node 22, instala dependencias com npm ci, roda testes headless e gera build de producao.

Na apresentacao, explique que qualidade nao depende apenas de clicar manualmente na tela. O projeto tem testes automatizados em diferentes niveis e pipeline para validar em ambiente limpo.

Capítulo 37 - Fluxo completo de criacao de corrida para apresentar

Para apresentar a criacao de corrida, comece pela tela do cliente. O cliente preenche origem e destino no formulario de nova corrida. O componente ClientRidesComponent valida os campos e chama RideService.createRide. Esse service monta uma chamada HTTP para o endpoint /rides usando a URL base do Gateway.

A requisicao chega ao API Gateway em localhost:8080. Como a rota comeca com /rides, o Gateway encaminha para o Ride Service. O RideController recebe CreateRideRequest, monta CreateRideCommand e chama DefaultCreateRideUseCase. O caso de uso cria a entidade Ride com status CREATED e driverId nulo.

Depois, o caso de uso consulta o Account Service usando AccountClient, cuja implementacao usa Feign. Essa etapa confirma que o cliente existe. Se o cliente nao existir, a criacao falha. Se existir, a corrida e salva no PostgreSQL por meio de RideGateway, RidePostgresGateway, RideEntity e RideRepository.

Com a corrida salva, o caso de uso publica RideCreatedMessage usando SentEventService. A implementacao envia para Kafka pelo RideProducer. O RideConsumer escuta o topico ride-topic, recebe a mensagem e chama DriverNotifier. A implementacao WebSocketDriverNotifier envia a notificacao para /topic/rides.

No frontend do motorista, RideNotificationService esta inscrito nesse topico. Quando a mensagem chega, DriverAvailableRidesComponent transforma a notificacao em RideDto e adiciona a corrida na lista de disponiveis. O motorista ve a nova corrida em tempo real.

Essa explicacao e uma das mais importantes do projeto, porque passa por quase todas as tecnologias. Ela mostra Angular, Gateway, Ride Service, Feign, Account Service, PostgreSQL, Kafka, WebSocket e tela do motorista.

Capítulo 38 - Fluxo completo de aceite de corrida para apresentar

Para apresentar o aceite de corrida, comece pela tela do motorista. O motorista visualiza uma corrida disponível e clica em aceitar. O componente `DriverAvailableRidesComponent` pega o id da corrida e o id da conta atual. Depois chama `RideService.acceptRide`, enviando uma requisicao POST para `/rides/{id}/accept`.

A requisicao passa pelo Gateway e chega ao `RideController`. O controller cria `AcceptRideCommand` com `rideId` e `driverId`. Depois chama `DefaultAcceptRideUseCase`. O caso de uso consulta o `Account Service` para validar se o motorista existe. Em seguida, verifica se o tipo da conta é `DRIVER`. Essa validacao é fundamental, porque impede um cliente de aceitar corrida.

Depois o caso de uso busca a corrida. Se ela não existir, responde como recurso não encontrado. Se existir, verifica se o status ainda é `CREATED`. Se a corrida já foi aceita, cancelada ou finalizada, o sistema impede novo aceite. Se estiver disponível, chama `assignDriver` na entidade `Ride`. Esse método atribui o motorista, muda status para `IN_PROGRESS` e atualiza a data.

A corrida atualizada é salva no PostgreSQL. Depois o sistema grava o status no Redis para consultas rápidas. Em seguida, envia notificacao via WebSocket. A tela do motorista remove a corrida da lista de disponíveis, porque ela já não está mais aberta para aceite.

Na apresentacao, destaque que o aceite tem regra de negocio importante. Não basta atualizar uma tabela. O sistema precisa validar perfil, validar status, salvar o estado, atualizar cache e avisar as telas.

Capítulo 39 - Fluxo completo de timeout para apresentar

O timeout existe para impedir que corridas fiquem abertas para sempre. A configuracao do Ride Service define quantos segundos uma corrida pode ficar aguardando aceite e de quanto em quanto tempo o job deve verificar. No projeto, a configuracao padrao usa cento e vinte segundos para timeout e trinta segundos para intervalo de verificacao.

RideTimeoutJob e um componente agendado. A anotacao Scheduled faz o metodo execute rodar automaticamente. Ele calcula uma data limite subtraindo o timeout do horario atual. Depois cria TimeoutRidesCommand e chama TimeoutRidesUseCase.

DefaultTimeoutRidesUseCase busca corridas expiradas usando RideGateway.findCreatedBefore. A implementacao em RidePostgresGateway chama

RideRepository.findTop50ByStatusAndCreatedAtBefore, procurando corridas com status CREATED e data de criacao anterior ao limite. Para cada corrida encontrada, o caso de uso muda o status para CANCELLED, salva no banco, atualiza Redis e notifica via WebSocket.

Esse fluxo mostra que o sistema nao depende apenas de acoes do usuario. Ele tambem executa processos internos automaticamente. Isso e comum em sistemas reais, onde tarefas agendadas limpam dados, expiram pedidos, enviam notificacoes ou geram relatorios.

Na apresentacao, diga que o timeout protege a consistencia da experiencia. Uma corrida que nunca foi aceita nao deve continuar aparecendo como disponivel indefinidamente. O job automatiza esse controle e mantem banco, cache e frontend sincronizados.

Capítulo 40 - Como explicar arquitetura limpa sem usar termos difíceis

Se a banca for iniciante, explique a arquitetura dizendo que o projeto separa responsabilidades. Uma parte representa as regras do negócio. Outra parte executa ações do sistema. Outra parte conversa com tecnologias externas. Essa explicação é suficiente para mostrar organização sem usar muitos termos técnicos.

Se a banca tiver conhecimento intermediário, diga que o projeto está dividido em domain, application e infrastructure. O domain contém entidades, enums e interfaces. O application contém casos de uso. A infrastructure contém controllers, banco, Kafka, Redis, WebSocket, Feign e configurações. Essa divisão evita misturar regra com detalhes de tecnologia.

Se a banca perguntar de forma avançada, explique inversão de dependência. A regra de negócio depende de contratos, não de implementações concretas. `DefaultCreateRideUseCase` depende de `RideGateway`, `AccountClient` e `SentEventService`. Ele não depende de `JpaRepository`, `FeignClient` ou `KafkaTemplate`. Isso torna a regra mais testável e flexível.

Use exemplos simples. `RideGateway` é o contrato que diz que o sistema precisa salvar e buscar corridas. `RidePostgresGateway` é a implementação que faz isso usando PostgreSQL.

`DriverNotifier` é o contrato que diz que o sistema precisa notificar motoristas.

`WebSocketDriverNotifier` é a implementação que faz isso usando WebSocket.

Essa forma de explicar mostra maturidade. Você não precisa falar apenas nomes de tecnologias. Você mostra por que o código foi organizado desse jeito e como isso ajuda testes, manutenção e evolução.

Capítulo 41 - Perguntas prováveis e respostas em forma de explicação

Se perguntarem por que usar Kafka se dava para chamar WebSocket diretamente, responda que em um projeto pequeno seria possível chamar WebSocket direto, mas Kafka desacopla a criação da corrida da notificação. O caso de uso publica um evento, e qualquer interessado pode reagir a esse evento. Hoje o consumidor envia notificação. No futuro, outro consumidor poderia gerar auditoria, métricas ou histórico sem alterar a criação da corrida.

Se perguntarem por que usar Redis se o dado já está no PostgreSQL, explique que Redis é cache, não fonte principal. Ele guarda uma cópia temporária do status para leitura rápida. Se o cache não tiver o dado, o sistema consulta o PostgreSQL. Assim, o banco continua sendo a fonte persistente e o Redis apenas melhora consulta.

Se perguntarem se a sessão do frontend é autenticação real, responda que não. A sessão com localStorage simula seleção de perfil para o desafio. Ela permite demonstrar cliente e motorista sem implementar login completo. Em uma evolução real, seria necessário usar autenticação com senha, token JWT, refresh token e autorização no backend.

Se perguntarem sobre dois motoristas aceitando ao mesmo tempo, explique que o caso de uso valida se a corrida ainda está CREATED antes de aceitar. Para um ambiente de alta concorrência real, você reforçaria essa regra com lock otimista, versionamento ou update condicional no banco. Isso mostra que você entende a solução atual e também sabe como evoluir.

Se perguntarem o que você melhoraria, responda que adicionaria autenticação real, autorização no backend, migrações com Flyway, paginação, lock de concorrência no aceite, observabilidade com logs estruturados e tracing, além de outbox pattern para eventos mais confiáveis.

Capítulo 42 - Roteiro de fala para apresentacao

Voce pode abrir dizendo que o Ride Challenge e uma aplicacao full stack de corridas, com frontend em Angular e backend em microservicos Java usando Spring Boot e Spring Cloud. O objetivo e simular um fluxo realista em que clientes criam corridas, motoristas recebem essas corridas em tempo real e podem aceitar uma delas.

Depois explique a arquitetura geral. Diga que o frontend chama o API Gateway. O Gateway roteia para Account Service ou Ride Service. O Config Server centraliza configuracoes. O Eureka permite que os servicos se encontrem pelo nome. O PostgreSQL guarda dados persistentes. O Kafka transporta eventos. O Redis guarda cache de status. O WebSocket atualiza a tela do motorista em tempo real.

Em seguida, fale do Account Service. Explique que ele gerencia contas de clientes e motoristas. A entidade Account valida dados. Os casos de uso criam, listam e buscam contas. A infraestrutura expoe endpoints REST e salva no PostgreSQL.

Depois fale do Ride Service. Explique que ele e o nucleo do sistema. Ele cria corrida, valida cliente consultando Account Service, salva no banco, publica evento no Kafka, consome evento, notifica motoristas, aceita corrida, atualiza Redis e cancela corridas antigas por timeout.

Depois mostre o frontend. Explique que Angular organiza rotas e componentes. A tela de login seleciona perfil. A tela do cliente cria e edita corridas. A tela do motorista lista corridas disponiveis e recebe notificacoes WebSocket. Os services Angular chamam a API e os DTOs tipam os dados.

Feche falando de qualidade. Diga que o projeto tem testes unitarios no backend com JUnit e Mockito, testes de integracao com Testcontainers, testes no frontend com Jasmine e Karma, smoke E2E com Cypress, collection Postman e pipeline de CI com GitHub Actions.

Capítulo 43 - Explicação final para memorizar

Se você precisar resumir o projeto inteiro em uma fala curta, diga que o sistema foi criado para demonstrar um fluxo de corridas em tempo real. O cliente cria uma corrida no Angular. A chamada passa pelo API Gateway e chega ao Ride Service. O Ride Service valida a conta no Account Service, salva a corrida no PostgreSQL e publica um evento no Kafka. O consumidor Kafka recebe esse evento e usa WebSocket para avisar os motoristas conectados. Quando um motorista aceita, o backend valida se ele é motorista, muda o status da corrida, salva no banco, atualiza Redis e notifica novamente.

Se quiser resumir as tecnologias, diga que Angular entrega a interface, Java e Spring Boot entregam os microsserviços, Spring Cloud Config centraliza configuração, Eureka faz descoberta de serviços, Gateway centraliza entrada, PostgreSQL persiste dados, Kafka trabalha com eventos, Redis atua como cache, WebSocket entrega tempo real, Docker Compose sobe o ambiente e os testes garantem qualidade.

Se quiser resumir a arquitetura, diga que o código foi dividido em camadas. O domínio contém as regras e contratos. A aplicação contém os casos de uso. A infraestrutura contém detalhes externos. Essa separação torna o projeto mais organizado, mais testável e mais fácil de evoluir.

Se quiser fechar de forma forte, diga que o principal valor do projeto é integrar várias tecnologias com responsabilidades claras. Ele não é apenas um CRUD. Ele mostra fluxo em tempo real, microsserviços, evento assíncrono, cache, persistência, Docker e testes automatizados trabalhando juntos.

Essa é a ideia que você deve levar para a apresentação. Não tente decorar cada linha. Entenda a função de cada parte e explique como elas se conectam. Quando você entende o caminho da corrida, desde o clique no frontend até a notificação no motorista, você consegue responder a maioria das perguntas.